

BIND MOUNTS:

A **bind mount** we can share a file or directory from the host machine into a container.

Syntax: `-v <path-in-dockerhost>:<path-in-container>`

Using bind mount we can mount any file folder in to container

Practicals:

Create my-app-directory on dockerhost

```
ubuntu@ip-172-31-25-109:~$  
ubuntu@ip-172-31-25-109:~$ mkdir my-app-directory  
ubuntu@ip-172-31-25-109:~$ ls  
my-app-directory  
ubuntu@ip-172-31-25-109:~$
```

Create a container & also map **my-app-directory** in to container inside `/opt/my-app-directory` directory & create 5 files as shown below

```
ubuntu@ip-172-31-25-109:~$ docker run --name c1 -v /home/ubuntu/my-app-directory:/opt/my-app-directory -it amazonlinux  
bash-5.2#  
bash-5.2#  
bash-5.2# cd /opt/my-app-directory/  
bash-5.2#  
bash-5.2# touch file1 file2 file3 file4 file5  
bash-5.2#  
bash-5.2# ubuntu@ip-172-31-25-109:~$  
ubuntu@ip-172-31-25-109:~$
```

Come out of container to docker host & check the files created in container are available in `/home/ubuntu/my-app-directory`

```
ubuntu@ip-172-31-25-109:~$ cd my-app-directory/  
ubuntu@ip-172-31-25-109:~/my-app-directory$ ls  
file1 file2 file3 file4 file5  
ubuntu@ip-172-31-25-109:~/my-app-directory$
```

Now even if container c1 gets deleted data will be preserved in dockerhost

Docker volumes:

Why do we need volumes?

we have to use volumes because if a container exits / deleted then all files present in the container gets deleted.

so to store (persist) few important files present in container into Dockerhost, so that even if a container gets deleted, we will have all those important files stored in docker host .

Also, with volumes we can share files in one container with other containers also.

Important pointers on volumes:

- ❖ Docker volume is simply a directory inside our container.
- ❖ we can declare a directory as a volume ONLY WHILE CREATING THE CONTAINER.

- ❖ we can't create volume from an already running container.
- ❖ we can share *same volume* across any number of containers.
- ❖ we can map volume in two ways
 - Host $\longleftrightarrow$ Container
 - Container $\longleftrightarrow$ Container

Benefits of Volume:

- ❖ Decoupling storage from containers.
- ❖ Share volume among different containers.
- ❖ On deleting the container, the volume does not delete.

Working with volumes:

Step1. create a volume

syntax: `docker volume create <volume_name>`
`docker volume create my-volume-1`

#observation: run `docker volume ls` & see `myvol1` gets created

Step 2. let's assume important directory in our container is **/opt/my-app-dir-2**, now to preserve files present in this directory into docker host

create a container with volume mounting along with docker run command

syntax: `-v <volumeName>:<dir_path_in_container>`

syntax: `docker run --name c1 -v <volume_name>:<dir_path_in_container> -it ubuntu`

we will be logged into newly created container, now let's go inside the `/opt/myimportantdirectory/` & lets create few files `myimpfile1.log`, `myimpfile2.log`, `myimpfile3.log`, `myimpfile4.log` & `myimpfile5.log`

```
ubuntu@ip-172-31-25-109:~$ docker run --name c1 -v my-volume-1:/opt/my-app-dir-2 -it ubuntu
root@4da54af01ce1:/#
root@4da54af01ce1:/# cd /opt/my-app-dir-2/
root@4da54af01ce1:/opt/my-app-dir-2#
root@4da54af01ce1:/opt/my-app-dir-2# ls
root@4da54af01ce1:/opt/my-app-dir-2# touch myimpfile{1..5}.log
root@4da54af01ce1:/opt/my-app-dir-2# ls
myimpfile1.log myimpfile2.log myimpfile3.log myimpfile4.log myimpfile5.log
root@4da54af01ce1:/opt/my-app-dir-2#
root@4da54af01ce1:/opt/my-app-dir-2# ubuntu@ip-172-31-25-109:~$
ubuntu@ip-172-31-25-109:~$
```

press `ctrl p+q` ==> come out of container & back to dockerhost

Step 3. verify the files present in docker host as in containers directory

Docker volumes will be stored in -> `/var/lib/docker/volumes/<volumeName>/_data`

```
ubuntu@ip-172-31-25-109:~$ sudo su -
root@ip-172-31-25-109:~# cd /var/lib/docker/volumes/my-volume-1/_data/
root@ip-172-31-25-109:/var/lib/docker/volumes/my-volume-1/_data#
root@ip-172-31-25-109:/var/lib/docker/volumes/my-volume-1/_data# ls
myimpfile1.log myimpfile2.log myimpfile3.log myimpfile4.log myimpfile5.log
root@ip-172-31-25-109:/var/lib/docker/volumes/my-volume-1/_data#
root@ip-172-31-25-109:/var/lib/docker/volumes/my-volume-1/_data# |
```

#observations: now try deleting c1 container we will still have all files present in /var/lib/docker/volumes/my-volume-1/_data directory

Step 4. With volume mounts we can share the same volume (my-volume-1) with multiple other containers also:

let's create a new container c3 from centos image use volume (my-volume-1) mounted to /opt/mydir1 inside the container

```
ubuntu@ip-172-31-25-109:~$ docker run --name c3 -v my-volume-1:/opt/mydir1 -it centos
[root@934761ad18b2 /]#
[root@934761ad18b2 /]#
[root@934761ad18b2 /]# cd /opt/mydir1/
[root@934761ad18b2 mydir1]# ls
myimpfile1.log myimpfile2.log myimpfile3.log myimpfile4.log myimpfile5.log
[root@934761ad18b2 mydir1]#
[root@934761ad18b2 mydir1]#
[root@934761ad18b2 mydir1]# |
```

#observations: after container gets created go to mounted directory i.e (/opt/mydir1) & list file, you can see all files from volume (my-volume-1) will be available inside c3 container also

+++++

Q: How do you list all Docker volumes on your system?

Use the command `docker volume ls`.

Q: How do you inspect a Docker volume?

Use the command `docker volume inspect <volume_name>`.

Q: How do you remove a Docker volume?

Use the command `docker volume rm <volume_name>`.

Q: Can you share Docker volumes between multiple containers?

Yes, volumes can be shared between multiple containers by specifying the same volume name when creating new container.

Q: Difference between bind mounts & docker volumes?

Feature	Bind Mounts	Volumes
Managed by Docker	No	Yes
Storage Location	any path can be given in dockerhost	Docker-managed directory /var/lib/docker/Volumes/<volume-name>
Use Case	Development, real-time file sync	Persistent storage, production use
Portability	Host-dependent	Portable and easier to back up
Usage	Less preferred	More preferred

Docker Images:

we can get docker images in 2 ways

- ♦ **Pre-defined images**
 - images which we download from Dockerhub
 - docker images in Dockerhub are prepared & maintained by docker company and docker community
- ♦ **Custom docker images**
 - custom images created by Devops engineers / developers of any company.
 - We can create docker image from a file called as **Dockerfile**. this is the most preferred way of creating docker images.

+++++

Can a custom Docker image be created from a running container?

Yes , we can create a image from any running container

Customization – means we can create some important files, install some of the softwares

Creating a custom image from any running container?

-
- create a container
 - do your customizations
customizations ==> install any software's (or) add files / directories
 - docker diff <ContainerID-or-ContainerNAME>
docker diff command shows custom changes we have made in the container in comparison with original image.
 - docker commit <container_name> <new_image_name>:<tag>
docker commit command will create image from the container name specified

```
ubuntu@ip-172-31-25-109:~$ docker run --name c1 -it alpine
/#
/# mkdir dir1 dir2 dir3
/# ubuntu@ip-172-31-25-109:~$
ubuntu@ip-172-31-25-109:~$
ubuntu@ip-172-31-25-109:~$ docker diff c1
A /dir3
A /dir1
C /root
A /root/.ash_history
A /dir2
ubuntu@ip-172-31-25-109:~$
ubuntu@ip-172-31-25-109:~$
ubuntu@ip-172-31-25-109:~$ docker commit c1 my-custom-alpineimage:v1
sha256:0ff60e910cb6dc17cca10c05113882144436802e3ca18bbfb4d413ed30c7b17c
ubuntu@ip-172-31-25-109:~$
ubuntu@ip-172-31-25-109:~$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
my-custom-alpineimage	v1	0ff60e910cb6	6 seconds ago	7.83MB

Dockerfile:

It is a simple text file, which uses predefined keywords or instructions for creating customized docker images.

- ❖ Docker will process Dockerfile instructions from top to bottom.
- ❖ If we build the Dockerfile, we can create a customize image
- ❖ If we run the created image, container will gets created
- ❖ In Dockerfile the first letter should be 'D'
- ❖ All instructions in Dockerfile starts will be in capital letter
- ❖ It's recommended to keep only one Dockerfile / directory

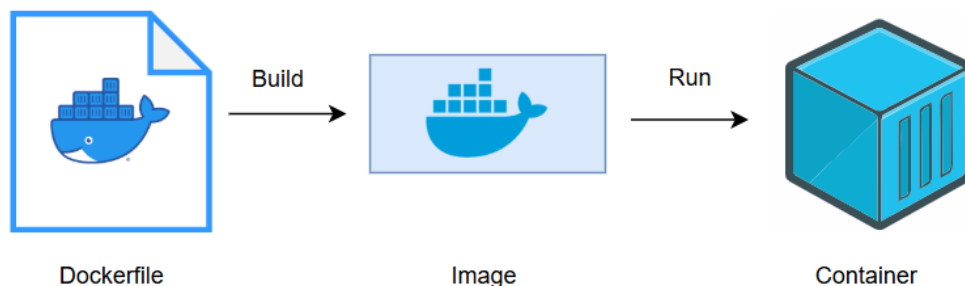
Steps involved in creation of custom docker image using Dockerfile:

Step 1: create a file named Dockerfile

Step 2: Add instructions in Dockerfile

Step 3: Build Dockerfile to create image

Step 4: Run the created image to create new container



Key words or instructions used in docker file (case sensitive)

FROM:

FROM instruction is used to represent base image to create our image.
On Top of base image our image will be created.

MAINTAINER:

it represents name of the organization or the author who created this Dockerfile.

RUN:

Used for running linux commands *while creating image*.

It is used to install packages, set up dependencies, and perform other build-time actions.

We can write multiple RUN instructions, and the docker will process all the RUN instructions from top to bottom.

CMD:

This is used to specify the initial command that should be executed when the container starts.

CMD instructions will be executed **when a container gets created**.

We cannot use this to execute multiple Commands.

If we define multiple CMD instructions, then the last CMD is always executed.

****Important_interview_Question****

What is difference between RUN & CMD in dockerfile?

Both RUN & CMD instructions are used to run Linux commands only but

RUN instructions will get executed while image is getting created

CMD instruction will get executed as initial command (first command) when a container gets created.

We can write multiple RUN instructions in Dockerfile, docker will process all those instructions one by one.

If we write multiple CMD instructions in Dockerfile, docker will process only the last CMD instruction.



Scenario 1:

Create a docker image named myimage & tag as v1, take ubuntu as the base image and specify the maintainer as bharath. Construct an image from the dockerfile.

vi Dockerfile

```
FROM ubuntu
MAINTAINER bharath
```

inorder to create a image from Dockerfile, we have to use docker build command like below

```
docker build -t <image_name>:<tag_name> .
```

```
docker build -t myimage:v1 .
```

Note:

. (dot) ==> dot in docker build command indicates the Dockerfile present in current directory

its always a best practice to keep Dockerfile in seperate directory
in dockerfiles indicates comment, these lines will be skipped from execution

Scenario 2:

Create a docker image named helloimage with tag as v3, take amazonlinux as the base image, create a file /etc/hellofile which has content "hello everyone".

vi Dockerfile

```
FROM amazonlinux
MAINTAINER bharath
RUN echo "hello everyone" > /etc/hellofile
```

#create a container from created image & go inside the container & cat /etc/hellofile to verify the content

Scenario 3:

Create a docker image named amazonlinuxgit with tag as 2023, take amazonlinux as the base image, i want git to be available in my container, Construct an image from the dockerfile.

Vi Dockerfile

```
FROM amazonlinux
RUN yum update -y
RUN yum install git -y
```

inorder to create a image from dockerfile, we have to use docker build command

docker build -t amazonlinuxgit:2023 .

#observation ==> create container, login & verify the git installation

Scenario 4:

create an image named importantImage with tag as v1
take ubuntu as the base image,
Whenever i create a container, i want date command / program to get executed first

vi Dockerfile

```
FROM ubuntu
MAINTAINER abc
```

CMD date

```
docker build -t importantImage:v1 .
```

```
ubuntu@ip-172-31-45-143:~$ docker run --name c5 -it importantImage:v1
```

```
Mon Oct 30 03:52:09 UTC 2023
```

```
ubuntu@ip-172-31-45-143:~$
```

Observation:

even though we have specified docker run command with -it container doesn't open interactive terminal, container just executed date command as mentioned in CMD instruction of Dockerfile & container gets moved into exited state.

what is the reason behind this?

+++++

Other Key words used in docker file.....< continued....>

ENTRYPOINT -- used to specify the default process that should be executed when container starts.

It can also be used for accepting arguments from the CMD instruction.

USER -- used to specify the default user who should login into the container.

WORKDIR -- Used to specify default working directory in the container

COPY -- Copying the files from the host machine to the container.

ADD -- Used for copying files from host to container, it can also be used for downloading files from remote servers.

ENV -- used for specifying the environment variables that should be passed to the container.

EXPOSE -- Used to specify the internal port of the container

VOLUME -- used to specify the default volume that should be attached to the container.

LABEL -- used for giving label to the container

Assignment:

- create different images with different Base image Flavors.
- investigate, why container stopped automatically after using CMD Instruction in senario4 ?
- difference between && (and operator) and || (or operator) used in linux.